Watopoly — Design Document 2

High Level Overview

The **types** module (`types.cc`) contains shared enums (`TurnPhase` , `PieceType` , `SLCEvent`) and small structs (`DiceResult`). It has zero dependencies on other project modules.

The **core** module (`core.cc` , `core-impl.cc`) contains the game's domain model: the `IGameContext` abstract class, `Player` , the `Square` hierarchy (all 11 concrete square types), the `Property` hierarchy (`AcademicBuilding` , `Residence` , `Gym`), and `MonopolyBlock` .

The **dice** module (`dice.cc` , `dice-impl.cc`) contains `RandomEventGenerator` (the centralized random number generator) and `Dice` (movement rolls with consecutive-doubles tracking).

The **board** module (`board.cc` , `board-impl.cc`) contains the `Board` class (which owns all 40 squares and 8 monopoly blocks) and the `buildDefaultBoard` factory function.

The **display** module (`display.cc` , `display-impl.cc`) contains the `DisplayObserver` abstract class, `DisplayHub` (observer management), `BoardDisplay` (the concrete text-based ASCII renderer), `EventLogger` (action history), and the snapshot structs (`GameSnapshot` , `SquareInfo` , `PlayerInfo`).

The **game** module (`game.cc` , `game-impl.cc`) contains `WatopolyGame` (the central controller that implements `IGameContext`), `CommandInterpreter` , the entire `Command` list, the `TurnContext` struct, and `WatopolyApp` (the command-line entry point).

The entry point is `main.cc` , which constructs a `WatopolyApp` and calls `run(argc, argv)` . `WatopolyApp` parses command-line flags (`-testing` , `-load` , `-seed` , `-enablelog` / `-enable-log` , `-enable-replay`) and delegates to `WatopolyGame` .

User Input Flow and Low Coupling / High Cohesion

User input arrives at `WatopolyGame::processTurn` , which reads a line from stdin and passes it to `CommandInterpreter::executeLine` . The interpreter looks up the command, checks `isValidInPhase(currentPhase)` , and calls the command's `execute` method. After every command, `WatopolyGame` publishes a `GameSnapshot` to all registered `DisplayObserver` instances using `DisplayHub` . This uses the Observer pattern.

This layering ensures that squares depend only on the `IGameContext` abstraction, the display depends only on snapshot data, and the command layer is the only part that touches `WatopolyGame` directly.

```
bool CommandInterpreter::executeLine(const std::string &line, WatopolyGame &game) {
    std::istringstream iss{line};
    std::string cmdName;
    if (!(iss >> cmdName)) return false;

    std::vector<std::string> args;
    std::string tok;
    while (iss >> tok) args.push_back(tok);

    Command *match = nullptr;
    int matchCount = 0;
    for (auto &[name, cmd] : commands_) {
        if (name.find(cmdName) == 0) {
            match = cmd.get();
            ++matchCount;
        }
    }

    if (matchCount == 0) {
        std::cout << "Unknown command: " << cmdName << std::endl;
        return false;
    }
    if (matchCount > 1) {
        // try exact match to resolve ambiguity
        auto it = commands_.find(cmdName);
        if (it != commands_.end()) {
            match = it->second.get();
        } else {
            std::cout << "Ambiguous command: " << cmdName << std::endl;
            return false;
        }
    }

    if (!match->isValidInPhase(game.turnContext().phase)) {
        std::cout << "Cannot use '" << match->cmdName() << "' right now." << std::endl;
        return false;
    }
    return match->execute(game, args);
}
```

This achieves low coupling (modules communicate through abstract interfaces and lightweight data types rather than concrete implementations) and high cohesion (each module has a single focused responsibility).

The overall program flow is as follows. The main function calls `WatopolyApp::run`, which parses flags and constructs `WatopolyGame`. It then calls either `setupNewGame` (which prompts for player count, names, and tokens, builds the board, and registers the display) or `loadGame` (which reconstructs state from a save file). Then the `run` method enters the main loop: while the game is not over, it calls `processTurn`, which announces the current player, handles the Tims Line if applicable, reads a command, dispatches it through the interpreter, and redraws the board.

When only one player remains, `announceWinner` is called.

Design

The `Square` hierarchy has 11 concrete classes. `Square` is the abstract base class with a pure virtual `landOn` method. `Property` is an abstract intermediate class that uses the Template Method Pattern. `Property` has three concrete subclasses: `AcademicBuilding` (which uses a tuition lookup table and supports improvements from 0 to 5, with a monopoly bonus), `Residence` (whose rent scales by count: $25, $50, $100, or $200), and `Gym`. The remaining eight concrete `Square` subclasses are: `CollectOSAP` (awards $200), `DCTimsLine` (`landOn` does nothing, since jailing is done via `sendPlayerToTims`), `GoToTims` (sends the player to Tims), `GooseNesting` (prints a flavour message), `TuitionSquare` (prompts a choice between $300 or 10% of total worth), `CoopFee` (charges $150), `SLCSquare` (checks for a 1% Roll Up the Rim cup first; if no cup is awarded, generates a random movement event), and `NeedlesHallSquare` (checks for a 1% Roll Up the Rim cup first; if no cup is awarded, generates a random money delta).

The `Command` abstract class has 12 concrete classes: `RollCommand`, `NextCommand`, `TradeCommand`, `ImproveCommand`, `MortgageCommand`, `UnmortgageCommand`, `BankruptCommand`, `AssetsCommand`, `AllCommand`, `SaveCommand`, `HistoryCommand`, and `ReplayCommand`.

The `Display` hierarchy has one concrete class: `BoardDisplay` (the text-based ASCII renderer), which extends the abstract `DisplayObserver` class.

Every class that is meant to be subclassed has a virtual destructor, as required by the course notes to prevent memory leaks during polymorphic deletion. All subclass overrides use the `override` keyword, which provides a compile-time check that the method being overridden is actually virtual in the superclass.

`IGameContext` is the central abstract class. It uses pure virtual methods to define the narrow set of operations that squares need from the game engine. `WatopolyGame` is the sole concrete subclass.

`Square` is an abstract class with a pure virtual `landOn(Player&, IGameContext&)`. Since there is no meaningful default implementation for "what happens when a player lands here," it is appropriate to make this a pure virtual method.

```
export class Square {
    std::string name_;
    int index_;
public:
    Square(std::string name, int index);
    virtual ~Square() = default;

    const std::string &name() const { return name_; }
    int index() const { return index_; }
    virtual void landOn(Player &player, IGameContext &game) = 0;
    virtual bool isProperty() const { return false; }
    virtual bool isAcademic() const { return false; }
};
```

`Property` is also an abstract class. It provides a concrete `landOn` implementation (the template method) and declares a pure virtual `calculateFee(Player&, IGameContext&)` that subclasses must override. This makes `Property` abstract because it still has an unimplemented pure virtual method, exactly as described in the course notes: "Any class that defines one or more pure virtual methods cannot be instantiated."

```
export class Property : public Square {
    Player *owner_ = nullptr;
    int purchaseCost_;
    bool mortgaged_ = false;
    bool transferredMortgaged_ = false;
public:
    Property(std::string name, int index, int cost);

    void landOn(Player &player, IGameContext &game) override;   // template method
    virtual int calculateFee(Player &visitor, IGameContext &game) = 0;  // pure virtual
    bool isProperty() const override { return true; }

    bool isMortgaged() const { return mortgaged_; }
    bool wasTransferredMortgaged() const { return transferredMortgaged_; }
    void setTransferredMortgaged(bool v) { transferredMortgaged_ = v; }
    void mortgage();
    void unmortgage();
};
```

The `transferredMortgaged_` flag tracks whether a property arrived in a player's portfolio while already mortgaged (via trade or bankruptcy transfer). `UnmortgageCommand` charges 60% of the purchase price for a normally mortgaged property and 70% for a transferred-mortgaged one.

`Command` is an abstract class with pure virtual `execute` and `isValidInPhase` methods. `DisplayObserver` is an abstract class with a pure virtual `update(const GameSnapshot&)` method.

Design Patterns

The `DisplayHub` class acts as the Subject. It maintains a vector of `DisplayObserver` pointers and provides `attach`, `detach`, and `publish` methods. `BoardDisplay` is the ConcreteObserver that implements the `update` method. After any state change, `WatopolyGame` calls `publish` on the `DisplayHub`, which iterates over all registered observers and calls `update` on each, passing a `GameSnapshot`.

```
export class DisplayObserver {
public:
    virtual ~DisplayObserver() = default;
    virtual void update(const GameSnapshot &snap) = 0;  // pure virtual
};

export class DisplayHub {
    std::vector<DisplayObserver *> observers_;
public:
    void attach(DisplayObserver *obs);
    void detach(DisplayObserver *obs);
    void publish(const GameSnapshot &snap);
};

void WatopolyGame::redraw() {
    displays_.publish(snapshot());
}
```

The `GameSnapshot` is a lightweight value struct containing `SquareInfo` for all 40 squares and a vector of `PlayerInfo`. This is equivalent to the getState pattern from the course notes.

Our implementation uses a factory function: `buildDefaultBoard(Board&)`. This free function constructs all 40 `Square` objects (with their correct names, positions, costs, tuition tables, and monopoly block assignments) and all 8 `MonopolyBlock` objects (Arts1, Arts2, Eng, Health, Env, Sci1, Sci2, Math), and places them in the `Board`. The `Board` class itself is a pure container with no knowledge of specific square types, and the game controller works only with `Square` pointers. This means changing board data (for example, rebalancing tuition values) or adding a new square type requires only changes to `buildDefaultBoard`, not to `Board` or the game controller.

The `CommandInterpreter` calls `command->execute(game, args)` through a `Command` pointer. The correct subclass implementation (`RollCommand`, `TradeCommand`, etc.) runs based on the dynamic type. This demonstrates proper use of polymorphism and virtual methods.

```
class Command {
public:
    virtual ~Command();
    virtual std::string cmdName() const = 0;
    virtual bool execute(WatopolyGame &game, const std::vector<std::string> &args) = 0;
    virtual bool isValidInPhase(TurnPhase phase) const = 0;
};
```

IGameContext

`IGameContext` is the central abstract class. It uses pure virtual methods to define the narrow set of operations that squares need from the game engine. `WatopolyGame` is the sole concrete subclass.

```
export class IGameContext {
public:
    virtual ~IGameContext() = default;

    // actions triggered by squares
    virtual void sendPlayerToTims(Player &player) = 0;
    virtual void promptPurchase(Player &player, Property &property) = 0;
    virtual void handleDebt(Player &debtor, int amount, Player *creditor) = 0;
    virtual void handleTuition(Player &player) = 0;
    virtual DiceResult rollForGym() = 0;
    virtual int activeCupCount() const = 0;
    virtual void awardCup(Player &player) = 0;
    virtual void resolveLanding(Player &player, int squareIndex) = 0;

    // event logging — called by square landOn implementations
    virtual void logEvent(const std::string &msg) = 0;

    // rng delegated from squares so core doesn't import dice
    virtual SLCEvent generateSLCEvent() = 0;
    virtual int generateNeedlesHallDelta() = 0;
    virtual bool checkRollUpTheRim() = 0;

    // state queries
    virtual TurnPhase getCurrentPhase() const = 0;
    virtual Player &currentPlayer() = 0;
    virtual std::vector<Player *> getActivePlayers() = 0;
    virtual Player *findPlayer(const std::string &name) = 0;
};
```

This follows the guidance to "program for interfaces instead of implementations" and to use abstract classes to provide method names and signatures that can be inherited and customized.

Our design achieves low coupling in several ways. Squares communicate with the game engine only through the `IGameContext` abstract class, which is "simple communication via parameters and results" (the lowest level of coupling described in the course notes). The display communicates through `GameSnapshot` value structs, not through direct access to `Board` or `Player` internals.

Our design achieves high cohesion because each module has parts that cooperate to perform one task. The core module contains only domain model classes (players, squares, properties). The dice module contains only randomness logic. The board module contains only the board container and its construction. The display module contains only presentation logic. The game module contains only orchestration and command dispatch. No module tries to do two unrelated things.

The game logic (the "Model" in MVC terms from the course notes) never prints directly to the user. Instead, it publishes snapshots through the Observer Pattern, and the `BoardDisplay` (the "View") handles all rendering. The `CommandInterpreter` and `WatopolyGame`'s turn loop serve as the "Controller."

Resilience to Change

If we wanted to add a new square type, we would only need to create a new subclass of `Square` (or `Property` if it is ownable), implement `landOn` (and `calculateFee` if applicable), and add it to `buildDefaultBoard`. This is because we have polymorphism.

If the command syntax changed, only the specific `Command` subclass's `execute` method that parses arguments would need to change. The `CommandInterpreter` dispatches by the first token and passes the rest as a vector of strings. Its parsing logic is unchanged, and other commands are unaffected.

If a game rule changed that is brokered by the game engine, the change would be made in `WatopolyGame`'s implementation of the relevant `IGameContext` method, or in the relevant square's `calculateFee`. No other `Square` or `Command` subclass needs modification. The abstract interface acts as a firewall between game-rule changes and the classes that trigger them.

If we wanted to add a graphical display, we would create a new `DisplayObserver` subclass and register it alongside `BoardDisplay` using `attach`. The game logic is completely unaware of how many or what kind of observers exist. The `GameSnapshot` struct already provides all the data a display needs.

If the save/load format changed, only `WatopolyGame`'s `saveGame` and `loadGame` methods would need to be updated. The game's internal representation is not coupled to the file format.

Player actions are separated from board representation. Changing game rules does not force changes to rendering. The display layer receives `GameSnapshot` objects and renders them with no knowledge of game rules, turn phases, or player actions. Similarly, changing the display format does not affect game logic.

If the input source changed (for example, reading from a file instead of stdin, or receiving commands over a network), the `CommandInterpreter` would remain unchanged. It already takes a string and dispatches it, regardless of where the string came from.

Answers to Questions

Q1: Would the Observer Pattern be a good pattern to use when implementing a game board? Why or why not?

This provides several concrete benefits. First, decoupling: game logic classes (`WatopolyGame` , `Player` , `Square` subclasses) do not need to know about the display at all. They modify state, and the Observer mechanism handles the rest. Second, extensibility: if we later wanted to add a graphical display, a logging observer, or a network observer, we simply create a new `DisplayObserver` subclass and register it using `attach` . Third, single responsibility: the `Board` class focuses on maintaining square data, and the `BoardDisplay` class focuses solely on rendering. Neither has to understand the other's internals. This is good for coupling and cohesion practices.

The main consideration is performance: redrawing a text board after every single command could produce a lot of output. However, since this is a turn-based game with human-speed input, this is not a practical concern.

Q2: Is there a suitable design pattern for modeling SLC and Needles Hall more closely to Chance and Community Chest cards?

The Strategy Pattern (or equivalently the Command Pattern applied to card effects) would be well-suited. Instead of hardcoding the probability distributions and effects directly inside `SLCSquare::landOn` and `NeedlesHallSquare::landOn` , we would model each possible outcome as its own class implementing a common `Card` abstract class with a pure virtual `execute(Player&, IGameContext&)` method and a description method.

Concrete cards would include `MoveForwardCard` , `MoveBackwardCard` , `GoToTimsCard` , `AdvanceToOSAPCard` , `GainMoneyCard` , `LoseMoneyCard` , and `RollUpTheRimCard` . A `CardDeck` class would hold a shuffled collection of `Card` objects. `SLCSquare` and `NeedlesHallSquare` would each own a `CardDeck` and, on `landOn` , draw the top card and call `card->execute(player, game)` .

This approach means adding new effects requires only a new `Card` subclass. Changing probabilities is a matter of adjusting card counts in the deck. Deck behaviour (draw-without-replacement, reshuffle when empty) can be encapsulated cleanly. The polymorphism means client code (`SLCSquare`) does not need to know which specific card was drawn; it just calls the virtual `execute` method, and the dynamic type determines the effect.

Q3: Is the Decorator Pattern a good pattern for implementing Improvements?

The Decorator Pattern is not a good fit for implementing improvements in Watopoly. The course notes describe the Decorator Pattern as a linked list of functionality, where each `ConcreteDecorator` wraps the next component and adds behaviour. This is useful when features can be composed in arbitrary combinations and toggled at runtime (like pizza toppings in the course notes example). Improvements in Watopoly do not have this structure:

First, improvements are uniform and countable. Each academic building has exactly 6 tuition tiers (0 to 5 improvements), and the tuition for each tier is a fixed value from a lookup table. Each "decorator" would do the same thing (change the tuition to the next table value), making the linked-list wrapping overhead pointless. Unlike the pizza example where each topping adds a different price and description, all improvements are identical.

Second, selling improvements requires unwrapping. Improvements can be sold in reverse order. With decorators, selling the most recent improvement means removing the outermost wrapper from the linked list, which is awkward. With a simple integer counter, selling is just decrementing the counter.

Third, querying the improvement count must be efficient. The board display, save/load, trade validation, and mortgage validation all need to quickly check how many improvements a building has. With decorators, this requires traversing the chain in O(n) time. With an integer, it is constant time O(1).

How these answers differ from Due Date 1

Our answers to the design questions are substantively unchanged from Due Date 1. The Observer Pattern answer is the same. The card-deck answer is the same. The Decorator answer is the same.

Extra Credit Features

Our implementation contains zero `delete` statements and zero raw `new` expressions. All dynamic memory is managed through RAII.

Players are stored as a `vector` of `unique_ptr<Player>` , indicating ownership and allowing polymorphism.

```
class WatopolyGame : public IGameContext {
    Board board_;
    std::vector<std::unique_ptr<Player>> players_;
    // ...
};
```

Squares are owned by `Board` via a `vector` of `unique_ptr<Square>` . Polymorphic destruction is handled automatically: because `Square` has a virtual destructor, the correct subclass destructor runs when the `unique_ptr` is destroyed, exactly as the course notes require ("if you know that a class may be subclassed, you must ALWAYS make its destructor virtual").

```
export class Board {
    std::vector<std::unique_ptr<Square>> squares_;
    std::vector<MonopolyBlock> blocks_;
    // ...
};
```

Commands are owned by `CommandInterpreter` via a `map` of string to `unique_ptr<Command>` .

```
class CommandInterpreter {
    std::map<std::string, std::unique_ptr<Command>> commands_;
public:
    void registerCommand(std::unique_ptr<Command> cmd);
    bool executeLine(const std::string &line, WatopolyGame &game);
};
```

`MonopolyBlock` s are stored by value in the `Board` 's vector.

Raw pointers in our project (`Player*` in `Property::owner_` , `MonopolyBlock*` in `AcademicBuilding` , `AcademicBuilding*` in `MonopolyBlock`) are used only where the pointed-to object has a guaranteed longer lifetime. Raw pointers represent non-ownership and do not free memory when they go out of scope.

This was challenging primarily in two areas. First, ensuring pointer stability during bankruptcy: when a player goes bankrupt, their properties are transferred to another player or auctioned. The bankrupt player's `unique_ptr` is not immediately removed from the vector (the player is marked bankrupt and skipped in turn order), so all `Player*` back-pointers remain valid. Second, the `MonopolyBlock` and `AcademicBuilding` cross-references: `Board` owns both, and they hold non-owning pointers to each other. The construction order in `buildDefaultBoard` must create `MonopolyBlock` s first, then create `AcademicBuilding` s with pointers to those blocks, then register the buildings back into the blocks. Since the `Board` 's block vector does not reallocate after construction, these pointers remain stable.

We implemented an `EventLogger` class that records a chronological log of game events. This is enabled via the `-enablelog` (or `-enable-log`) command-line flag. Two bonus commands were added: `history` (prints the last n events, or all events if no argument is given) and `replay` (displays replay events from a saved event stream). Adding them required only creating two new `Command` subclasses and registering them in `setupCommands` .

Final Questions

What lessons did this project teach you about developing software in teams?

Interface-first design is essential. Agreeing on the `IGameContext` abstract class and code signatures allowed all three of us to work independently on different modules. This connects directly to coupling: by communicating through abstract interfaces (the lowest level of coupling), our modules could be developed and tested independently.

The plan will change, and that is fine. Our Due Date 1 plan specified `SaveManager` , `Auction` , and `Trade` as standalone classes. During implementation, we discovered that inlining their logic was simpler and cleaner. The key was that we had a plan to deviate from. Without the DD1 design, we would not have had the structural understanding to know where simplification was safe.

What would you have done differently if you had the chance to start over?

First, we would keep commands on the `IGameContext` abstract class too. Our DD1 plan had commands depending on `IGameContext` , but we switched to `WatopolyGame` during implementation because commands needed access to functionality like `getDice` , `getBoard` , and `turnContext` . In retrospect, we should have extended `IGameContext` (or introduced a second abstract class like `ICommandContext`) to maintain the decoupling. The current design means adding a new command requires importing the game module, while adding a new square only requires importing core.

Second, we would design the snapshot-based Observer system from the start. The `GameSnapshot` and `DisplayHub` approach emerged during implementation when we realized that `Board` could not satisfy the original `DisplayObserver::update(Board&, vector<Player>&)` contract from the DD1 plan, because `Board` does not have access to player data. If we had designed the snapshot-based observer from the beginning, we would have avoided the architectural pivot mid-project.

Third, we would write integration test infrastructure earlier. We spent the first several days building features and only wrote tests near the end. Having a test harness with `-testing` mode and scripted input files from Day 2 would have caught bugs sooner and made integration smoother.

How the Design Changed from DD1

`SaveManager` was planned as a dedicated class for persistence. In the final implementation, save/load logic is inlined into `WatopolyGame` 's `saveGame` and `loadGame` methods. Introducing a separate class added indirection without meaningful benefit, since save/load needs access to `WatopolyGame` internals anyway.

`Auction` was planned as a standalone class owned by `WatopolyGame` . In the final implementation, auction logic lives in a `runAuction` method on `WatopolyGame` . The logic was simple enough that a dedicated class was over-engineering.

`Trade` was planned as a standalone class with `validate` , `presentTo` , and `execute` methods. In the final implementation, trade logic is inlined into `TradeCommand` 's `execute` method. As planned, trades are transient and stack-local (destroyed automatically when the command finishes, following RAII principles). In practice, the validation and execution logic fit naturally inside the command without a separate class.

`Board` was planned as the Observer Subject with `attach` , `detach` , and `notify` methods. In the final implementation, a separate `DisplayHub` class serves as the Subject, and `Board` is a pure container. `Board` had no access to player data needed by the display. Rather than passing player data through `Board` , we introduced `DisplayHub` as a standalone Subject that `WatopolyGame` owns.